Rollout Plan — Scaling DevSecOps Across 40 Microservices

Acme Financial Services | Prepared for: Engineering Manager | Companion to: Risk Brief, Pipeline Implementation (Juice Shop reference build)

The pipeline built for this engagement (SAST, secrets scanning, dependency/SBOM scanning, container scanning, and branch protection) works end-to-end on one application. This document covers how to scale the same set of controls to all 40 microservices without stalling the team's delivery pace. The technical rollout is the easy part — the organizational rollout, getting 15 developers and 3 DevOps engineers to actually adopt this without rejecting it, is the harder problem this plan is built to solve.

1. Phased Approach

Defining "critical service"

Before any phasing can happen, every one of the 40 services needs to be classified. A service is treated as **critical** if it meets any one of the following three criteria:

Criterion	Why it matters
Handles money movement or financial transactions	Direct financial and regulatory exposure — the clearest, highest-impact category.
Handles authentication or personal/sensitive data (PII)	A breach enables fraud and identity theft even without any money moving directly.
Internet-facing with no service in front of it	Directly reachable by an outside attacker; exposure alone raises risk independent of the other two criteria.

Three phases

Phase	Timeline	Scope	Gate mode
Phase 1 — Pilot	Weeks 1–2	3–5 services matching all three critical criteria (e.g. payment processing, authentication service)	Warn-only for all four scans
Phase 2 — Expand	Weeks 3–6	All remaining critical services (any one criterion) + all internet-facing services	Pilot services move to block; new services start warn
Phase 3 — Full rollout	Weeks 7–12	Remaining ~30 internal/lower-exposure services	SAST + secrets scanning move to block org-wide; container and dependency scanning move to block per service as triage completes

Rationale for starting narrow: a small pilot lets problems in the rollout process itself (unclear ownership, tooling friction, unrealistic thresholds) surface and get fixed on 3–5 services before they're multiplied across 40. Starting with the highest-risk services first, rather than the easiest ones, ensures the services that

would hurt the business most in a breach are protected soonest — even though they are also the services where teams will push back hardest, given how business-critical their delivery timelines usually are.

2. Adoption Strategy

Warn-only before block

Every gate, on every newly onboarded service, starts in **warn-only mode**: the scan runs and findings are visible in the PR, but nothing is blocked. This mirrors the distinction already built into the reference pipeline — the container scan's informational baseline job (exit-code: 0) versus the enforced gate (exit-code: 1). Warn-only gives each team time to triage their existing backlog of findings before the gate can block anything. Skipping this step means a team can be blocked from merging a one-line, unrelated fix because of pre-existing findings they didn't introduce — a fast way to make a team disable the gate entirely rather than live with it. Only once a team's active-finding count has been triaged down to near-zero does a gate flip to block mode for that service.

Metrics tracked

Metric	What it tells you
% of services onboarded (of 40)	Overall rollout pace — the simplest progress indicator for leadership reporting.
Mean time-to-fix for Critical/High findings	Whether teams are actually resolving what the gates surface, or letting it pile up.
Exceptions requested vs. granted vs. expired	Whether the exception process is being used for genuine, temporary cases or becoming a rubber stamp.
Developer-reported false-positive rate	Whether the tooling is trusted or being tuned out — the metric most likely to predict teams quietly routing around a gate.

These four are tracked together deliberately: onboarding percentage measures pace, time-to-fix measures effectiveness, and the exception/false-positive metrics measure whether teams are genuinely cooperating with the program or quietly routing around it. A rollout can look successful on pace alone while silently failing on the other two — tracking all four is what catches that.

3. Exception Handling

A developer who genuinely cannot fix a finding on the gate's timeline needs a real, fast process — not an informal workaround, and not a rubber stamp:

✓ **Request, in the PR itself:** the developer states which finding, why it can't be fixed now, and a proposed expiry date (e.g. "fixing next sprint, extend 2 weeks"). Keeping this in the PR, not a separate ticketing system, keeps friction low.

✓ **Single-reviewer approval:** one designated security reviewer approves or rejects, ideally same-day — slow approval pushes developers toward disabling the check themselves instead.

✓ **Time-boxed and centrally tracked:** exceptions auto-expire. A spreadsheet is sufficient at this stage for 40 services; the key property is automatic expiry, not the tooling sophistication.

✓ **Escalation on repetition:** if the same team requests exceptions for the same class of finding repeatedly, that escalates to their Engineering Manager rather than generating another exception — repetition signals a skills gap, a genuinely hard remediation, or a rule that needs tuning, not a pattern to keep approving around.

> The exception process exists for legitimate, temporary cases. If most PRs on a service are requesting exceptions, that is not evidence the process works — it is a signal the rollout for that service moved too fast, or the gate's thresholds need retuning, and warrants slowing down rather than approving through it.